



UNIVERSIDADE TÉCNICA DO ATLÂNTICO

Instituto de Engenharias e Ciências do Mar

Sidnei Cruz
scruz@uta.cv



Listas Ligadas

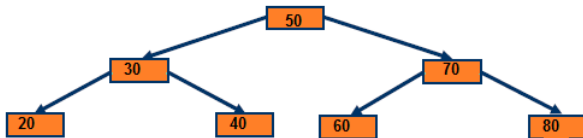
- ▶ As listas apresentam grandes vantagens sobre as sequências, por serem estruturas de dados dinâmicas e permitem o rearranjo eficiente dos seus elementos
- ▶ Embora tenham a vantagem de permitir a alocação dinâmica dos dados, as listas ligadas apresentam um problema:
 - ▶ Acesso sequencial
 - ▶ A pesquisa numa lista ligada e ordenada é feita percorrendo a lista desde o início até se encontrar o elemento pretendido.





Solução

- ▶ Conceber estruturas dinâmicas e ligadas que otimizem a pesquisa dos dados.
- ▶ É neste contexto que surgem as árvores. Para melhorar a pesquisa podemos ter uma estrutura que implemente o princípio da pesquisa binária.





Árvores são estruturas adequadas para representação de hierarquia.

- ▶ **Árvore:**
 - ▶ Um nó **c** é a RAIZ.





Árvores são estruturas adequadas para representação de hierarquia.

- ▶ **Árvore:**
 - ▶ Um nó **c** é a RAIZ.





Árvores são estruturas adequadas para representação de hierarquia.

- ▶ **Árvore:**
 - ▶ Um nó **c** é a RAIZ.
 - ▶ Todos os nós excepto **c** têm exactamente uma ligação para eles a partir de um nó **p**. **p** é o pai de **d**, **d** é o filho de **p**.





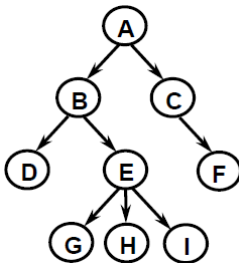
Árvores são estruturas adequadas para representação de hierarquia.

- ▶ **Árvore:**
 - ▶ Um nó **c** é a RAIZ.
 - ▶ Todos os nós excepto **c** têm exactamente uma ligação para eles a partir de um nó **p**. **p** é o pai de **d**, **d** é o filho de **p**.
 - ▶ Existe um caminho único da raiz **c** para cada nó. O número de ligações que têm de ser seguidas é o comprimento do caminho.



Árvores são estruturas adequadas para representação de hierarquia.

- ▶ **Árvore:**
 - ▶ Um nó **c** é a RAIZ.
 - ▶ Todos os nós excepto **c** têm exactamente uma ligação para eles a partir de um nó **p**. **p** é o pai de **d**, **d** é o filho de **p**.
 - ▶ Existe um caminho único da raiz **c** para cada nó. O número de ligações que têm de ser seguidas é o comprimento do caminho.





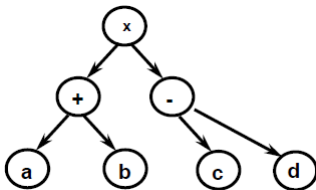
Aplicações

► Apl #1

Organização de dados, por exemplo que contêm vários níveis de informação (por exemplo, um edifício tem salas, uma sala tem objectos, os objectos podem ter outros objectos ...)

► Apl #2

Árvores de expressões. O valor de um nó é o resultado de aplicar o operador representado nesse nó utilizando como operandos os filhos desse nó





Aplicações

► Apl #3

Modelação de sistemas organizacionais (ex: famílias, directórios de um computador, hierarquia de comando de uma empresa,...).

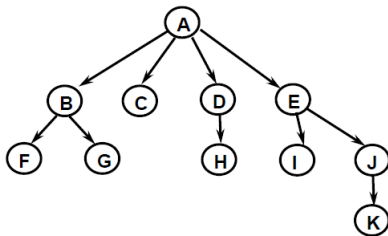




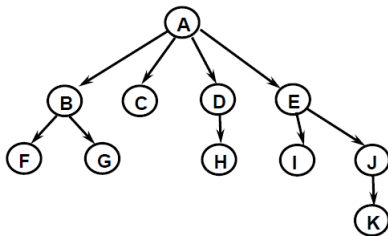
- ▶ **Altura de um nó (h)** - comprimento do caminho mais longo entre esse nó e as folhas da árvore (incluindo o próprio).
- ▶ **Profundidade de um nó** - comprimento do caminho da raiz até esse nó.
- ▶ **Tamanho de um nó** - número de descendentes desse nó mais ele próprio
- ▶ **Grau de um nó** - número de descendentes desse nó.
- ▶ **Grau de uma árvore** - é o máximo entre os graus de seus nós.
- ▶ **Floresta** - é um conjunto de zero ou mais árvores.



Exemplo:



Exemplo:



Nó	Altura	Profund.	Tam.
A	4	0	11
B	2	1	3
C	1	1	1
D	2	1	2
E	3	1	4
F	1	2	1
G	1	2	1
H	1	2	1
I	1	2	1
J	2	2	2
K	1	3	1

- ▶ **Árvore Binária:**
 - ▶ conjunto finito de elementos que
 - ▶ ou está **vazio**
 - ▶ ou contém um **elemento chamado raiz da árvore** ligado a **dois** conjuntos disjuntos em que cada um é, por sua vez uma **árvore binária**.

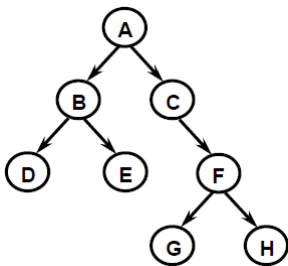


Figura: Árvore binária

- ▶ **Árvore Binária:**
 - ▶ conjunto finito de elementos que
 - ▶ ou está **vazio**
 - ▶ ou contém um **elemento chamado raiz da árvore** ligado a **dois** conjuntos disjuntos em que cada um é, por sua vez uma **árvore binária**.

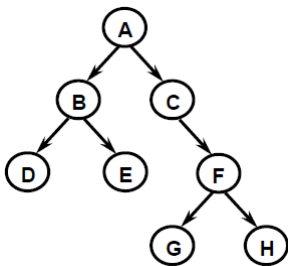


Figura: Árvore binária

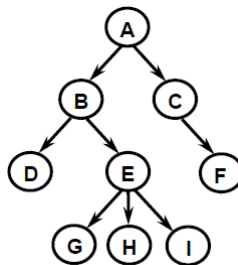
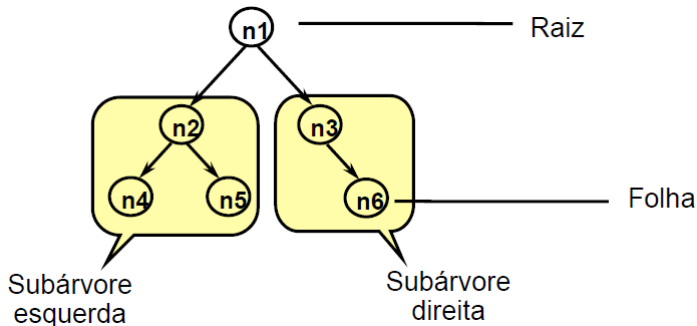


Figura: Árvore não binária



- ▶ Dois nós são irmãos se são os filhos direito e esquerdo do mesmo pai;
- ▶ Um nó que não tem filhos é uma folha;
- ▶ No exemplo acima, n4 e n5 são descendentes de n2 e este é ascendente de ambos.





- ▶ Nível de uma árvore binária:
 - ▶ Nível da raiz = 0;
 - ▶ Nível de qualquer outro nó = nível do pai + 1.

- ▶ Uma **árvore estritamente binária** é uma árvore binária na qual cada nó possui 0 ou 2 filhos.

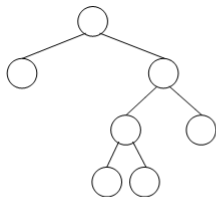




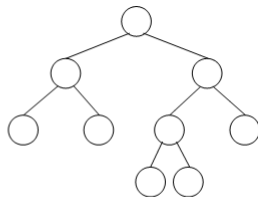
- ▶ Uma **árvore binária completa** é aquela que apresenta a seguinte propriedade: se v é um nó tal que alguma subárvore de v é vazia, então v se localiza ou no último ou no penúltimo nível da árvore
 - ▶ Só há nós com alguma subárvore vazia no penúltimo ou no último nível.
 - ▶ Altura mínima para um número de nós $n > 0$: $h = 1 + \lg n$
 - ▶ Ex.: $n = 8 \rightarrow h = 1 + \lg(8) = 1 + 3 = 4$
- ▶ Uma **árvore binária cheia** é aquela que se v é um nó tal que alguma subárvore de v é vazia, então v se localiza no último nível da árvore
 - ▶ Toda árvore binária cheia é **completa** e **estritamente binária**



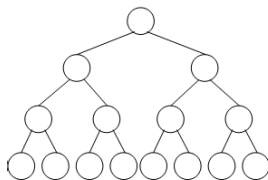
► Tipos especiais



estritamente binária



completa



cheia

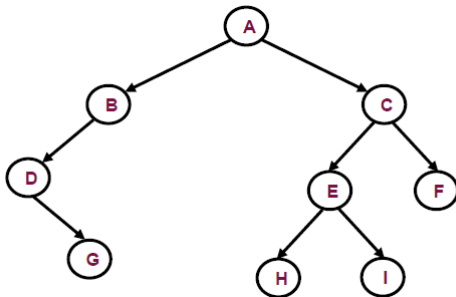


Travessia de uma árvore (cont.)

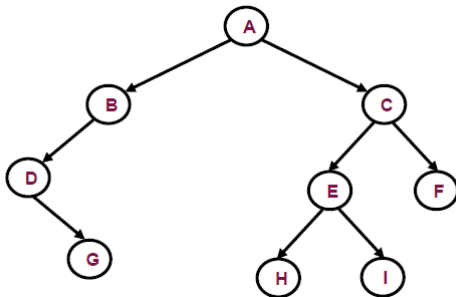
- ▶ Em **pré-ordem** (ou pré-fixado)
 1. Visitar a raiz;
 2. Atravessar a subárvore esquerda em pré-ordem;
 3. Atravessar a subárvore direita em pré-ordem.
- ▶ Em **ordem** (ou central)
 1. Atravessar a subárvore esquerda em ordem;
 2. Visitar a raiz;
 3. Atravessar a subárvore direita em ordem.
- ▶ Em **pos-ordem** (ou pos-fixado)
 1. Atravessar a subárvore esquerda em pos-ordem;
 2. Atravessar a subárvore direita em pos-ordem.
 3. Visitar a raiz;



Travessia de uma árvore - Exemplo

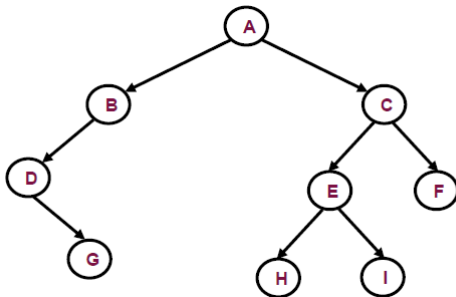


Travessia de uma árvore - Exemplo



Pré-ordem: A B D G C E H I F

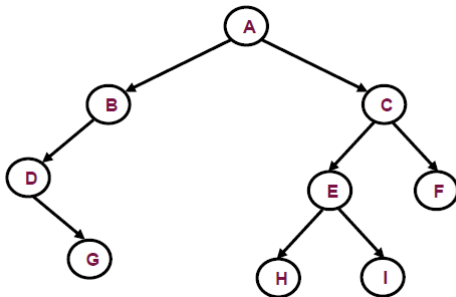
Travessia de uma árvore - Exemplo



Pré-ordem: A B D G C E H I F

Em ordem: D G B A H E I C F

Travessia de uma árvore - Exemplo

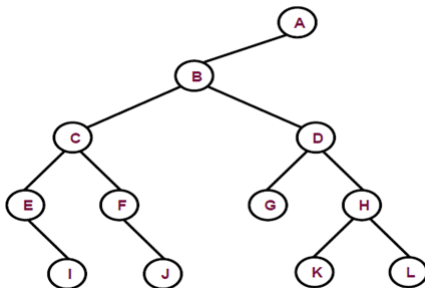


Pré-ordem: A B D G C E H I F

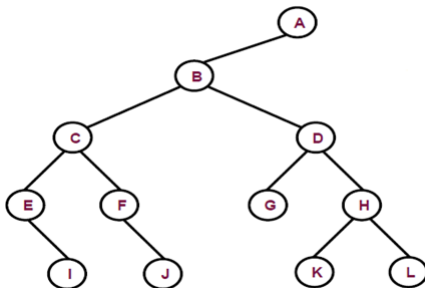
Em ordem: D G B A H E I C F

Pos-ordem: G D B H I E F C A

Travessia de uma árvore - Exemplo

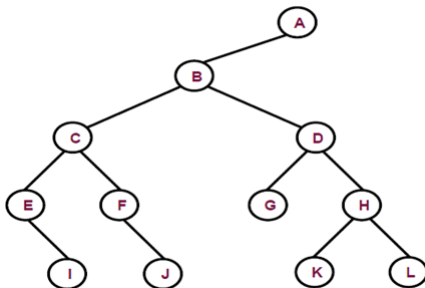


Travessia de uma árvore - Exemplo



Pré-ordem: A B C E I F J D G H K L

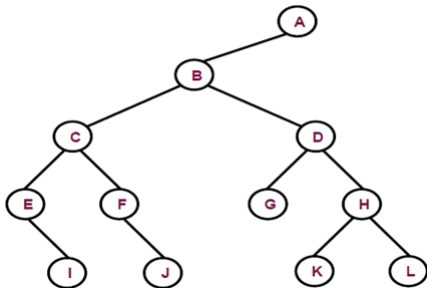
Travessia de uma árvore - Exemplo



Pré-ordem: A B C E I F J D G H K L

Em ordem: E I C F J B G D K H L A

Travessia de uma árvore - Exemplo



Pré-ordem: A B C E I F J D G H K L

Em ordem: E I C F J B G D K H L A

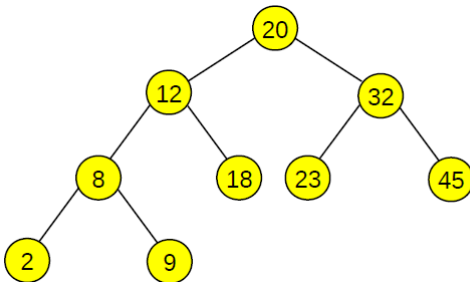
Pos-ordem: I E J F C G K L H D B A



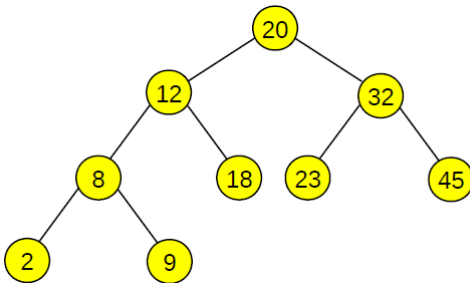
- ▶ Se os dados armazenados numa árvore binária obedecerem a algum critério de ordenação então diz-se que a árvore é uma árvore binária de pesquisa (em inglês BST - **Binary Search Tree**).



- ▶ Se os dados armazenados numa árvore binária obedecerem a algum critério de ordenação então diz-se que a árvore é uma árvore binária de pesquisa (em inglês BST - **Binary Search Tree**).



- ▶ Cada nó de uma **árvore binária de pesquisa** ou **BST** obedece às seguintes condições:
 - ▶ O valor armazenado no nó é maior do que todos os valores armazenados na sub-árvore da esquerda.
 - ▶ O valor armazenado no nó é menor do que todos os valores armazenados na sub-árvore da direita.
 - ▶ Ambas as sub-árvores da direita e da esquerda são também elas próprias árvores binárias de pesquisa.



- ▶ Principais operações definidas sobre uma árvore binária:
 - ▶ Inserção de um novo nó.
 - ▶ Eliminação de um nó.
 - ▶ Pesquisa de um nó.
 - ▶ Travessia da árvore por uma qualquer ordem.
- ▶ Utilizam-se muitas vezes procedimentos recursivos para implementar algumas destas operações.

► Definição da Estrutura de Dados:

```
typedef char telem;  
typedef struct no{  
    struct no* esq;  
    telem info;  
    struct no* dir;  
}tno;  
typedef tno* tarvbin;
```

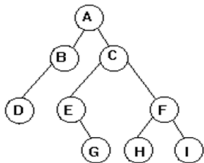


Figura: Árvore binária

► Definição da Estrutura de Dados:

```
typedef char telem;  
typedef struct no{  
    struct no* esq;  
    telem info;  
    struct no* dir;  
}tno;  
typedef tno* tarvbin;
```

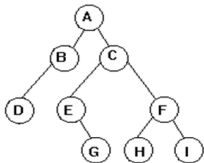


Figura: Árvore binária

► Definição da Estrutura de Dados:

```
typedef char telem;  
typedef struct no{  
    struct no* esq;  
    telem info;  
    struct no* dir;  
}tno;  
typedef tno* tarvbin;
```

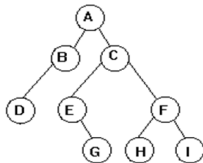


Figura: Árvore binária

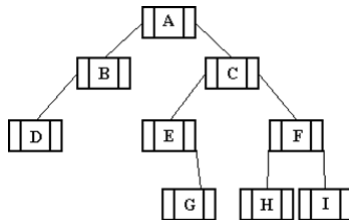


Figura: Árvore binária



▶ Criar uma árvore vazia

- ▶ Define uma árvore vazia e deve ser utilizado antes de qualquer operação na árvore.

```
void criar(tarvbin *T){  
  
    *T = NULL;  
  
}
```





▶ **Criar uma árvore vazia**

- ▶ Define uma árvore vazia e deve ser utilizado antes de qualquer operação na árvore.

```
void criar(tarvbin *T){  
  
    *T = NULL;  
  
}
```

▶ **Verifica se árvore vazia ou não**

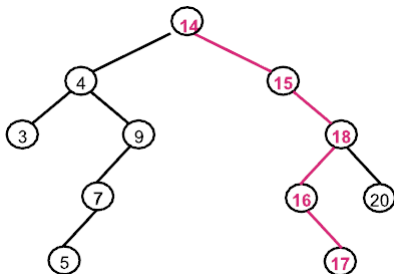
- ▶ Retorna 1 se árvore estiver vazia, 0 caso contrário.

```
int vazia(tarvbin *T){  
  
    return (T == NULL);  
  
}
```



- ▶ Método de Pesquisa:
 - ▶ Começar na raiz, repetindo o seguinte processo:
 - ▶ Se o elemento a procurar for igual ao nó actual parar: item encontrado, devolve verdadeiro.
 - ▶ Caso contrário e se o elemento a procurar for maior do que o nó actual então fazer o nó actual igual ao nó à direita.
 - ▶ Caso contrário fazer o nó actual igual ao nó da esquerda.
 - ▶ Se entretanto o nó actual passar a ser NULL, então não existe mais nenhum caminho a percorrer logo o elemento procurado não existe. Devolve falso.

► Método de Pesquisa:



- **Por exemplo**, para pesquisar o elemento 17 na árvore da figura seria apenas necessário percorrer os elementos assinalados.

► Versão Iterativa:

```
int procura(tarvbin Raiz, telem Item){  
  
    tarvbin Actual;  
    Actual=Raiz;  
  
    while (Actual != NULL){  
        if (Actual->info == Item)  
            return(1);  
        else{  
            if (Item > Actual->info)  
                Actual=Actual->dir;  
            else  
                Actual=Actual->esq;  
        }  
    }  
    return 0;  
}
```


- ▶ Método de Pesquisa (versão recursiva):
 - ▶ Condições de paragem:
 - ▶ Se a raiz é igual ao elemento a procurar então devolve verdadeiro
 - ▶ Se a raiz é NULL então o elemento a procurar não existe, logo devolve falso.
 - ▶ Caso geral:
 - ▶ Se o elemento a procurar for maior do que o nó actual então devolve procura na sub-árvore da direita.
 - ▶ Caso contrário devolve procura na sub-árvore da esquerda.



► Versão Recursiva:

```
int ProcuraRecursiva(tarvbin Raiz, telem Item){  
  
    tarvbin Actual;  
    Actual=Raiz;  
  
    if(Raiz==NULL)  
        return 0;  
    if(Raiz->info==Item)  
        return 1;  
    if (Item> Raiz->info)  
        return ProcuraRecursiva(Raiz->dir,Item);  
    else  
        return ProcuraRecursiva(Raiz->esq,Item);  
}
```



► Versão Recursiva:

► Procurar um elemento na árvore

- procura um elemento na árvore, retornando o seu endereço, caso o encontre. Se o elemento não for encontrado, retorna o endereço nulo (NULL).

```
tarvbin find(tarvbin T, telem dado){  
    tarvbin encontrou;  
    if(T == NULL)  
        return NULL;  
  
    if(T->info == dado)  
        return T;  
  
    encontrou = find(T->esq,dado);  
    if(encontrou == NULL)  
        encontrou = find(T->dir,dado);  
  
    return encontrou;  
}
```

▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

▶ Travessia em Pré-Ordem

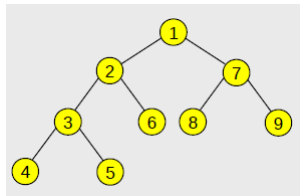
- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

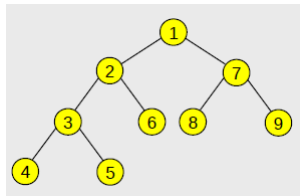


▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

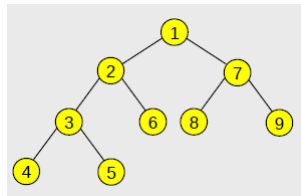
▶ 1



▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```



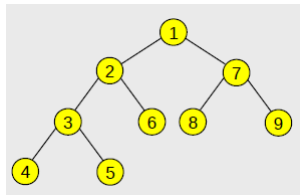
▶ 1 2

▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

▶ 1 2 3

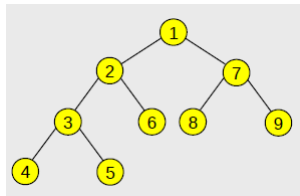


▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

▶ 1 2 3 4

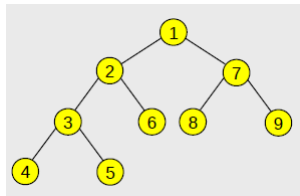


▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

▶ 1 2 3 4 5

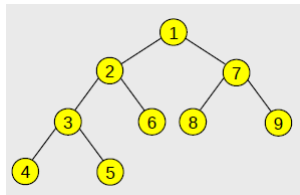


▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

▶ 1 2 3 4 5 6

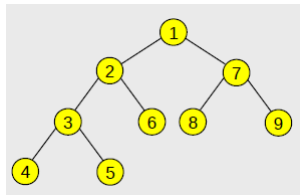


▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

▶ 1 2 3 4 5 6 7

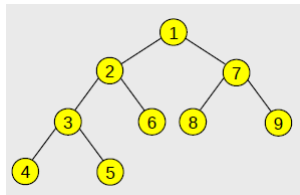


▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

▶ 1 2 3 4 5 6 7 8

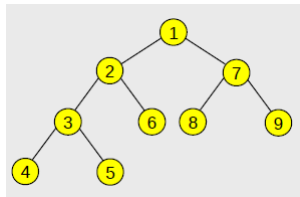


▶ Travessia em Pré-Ordem

- ▶ Visita raiz antes dos filhos

```
void EscrevePreOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    printf(" %c ",Raiz->info);
    EscrevePreOrdem(Raiz->Esq);
    EscrevePreOrdem(Raiz->Dir);
}
```

▶ 1 2 3 4 5 6 7 8 9



▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```


▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

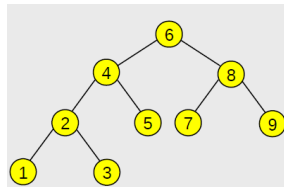
```
void EscreveOrdem(tarvbin Raiz)
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```

▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```

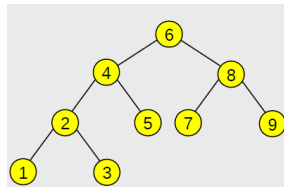


▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```



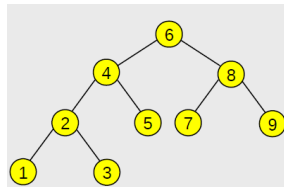
- ▶ 1

▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```



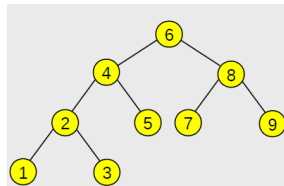
- ▶ 1 2

▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```



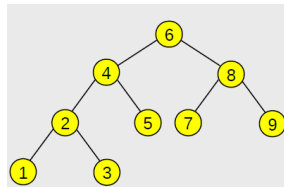
- ▶ 1 2 3

▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```



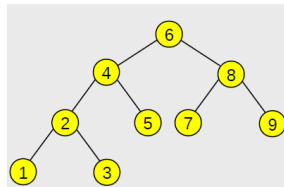
- ▶ 1 2 3 4

▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```



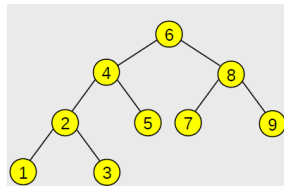
- ▶ 1 2 3 4 5

▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```



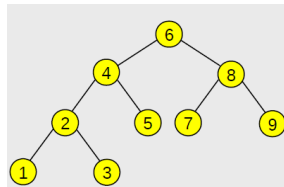
- ▶ 1 2 3 4 5 6

▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```



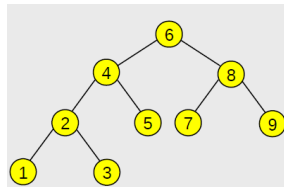
- ▶ 1 2 3 4 5 6 7

▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```



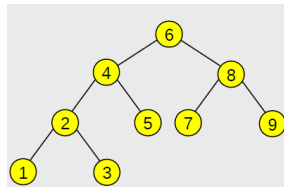
- ▶ 1 2 3 4 5 6 7 8

▶ Travessia em Em Ordem

- ▶ Visita a raiz depois do filho esquerdo e antes do direito

- ▶ Exemplo de aplicação: imprime os nós ordenados

```
void EscreveOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscreveOrdem(Raiz->Esq);
    printf(" %c ",Raiz->info);
    EscreveOrdem(Raiz->Dir);
}
```



- ▶ 1 2 3 4 5 6 7 8 9



▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos
- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```





▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos
- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```

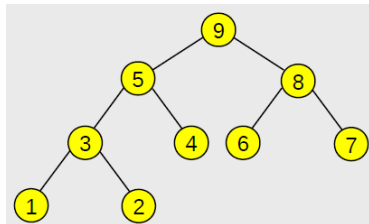


▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```

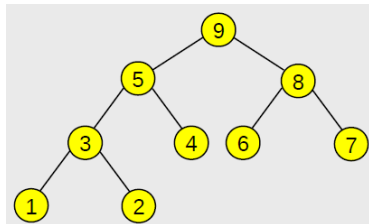


▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```



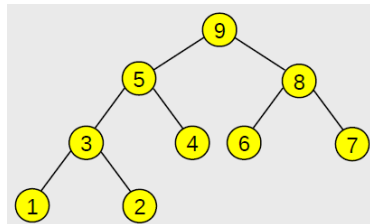
- ▶ 1

▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```



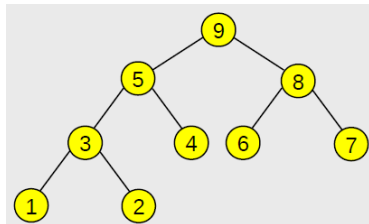
▶ 1 2

▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```



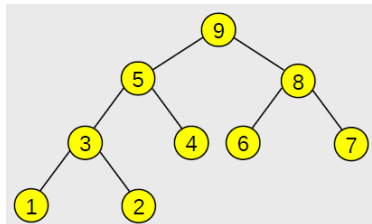
▶ 1 2 3

▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```



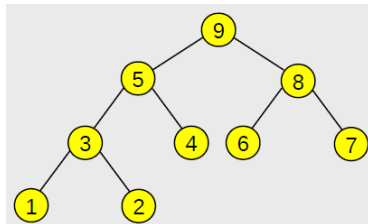
▶ 1 2 3 4

▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```



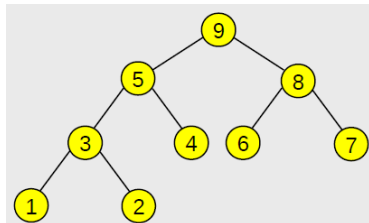
- ▶ 1 2 3 4 5

▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```



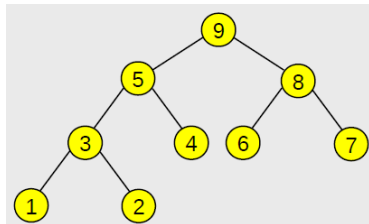
- ▶ 1 2 3 4 5 6

▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```



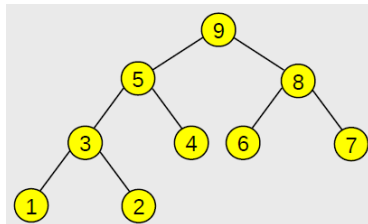
▶ 1 2 3 4 5 6 7

▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```



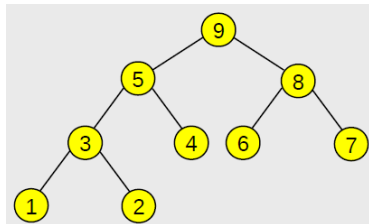
- ▶ 1 2 3 4 5 6 7 8

▶ Travessia em Pós-Ordem

- ▶ Visita a raiz depois dos filhos

- ▶ Exemplo de aplicação: avaliação de expressões posfixadas

```
void EscrevePosOrdem(tarvbin Raiz)
{
    if (Raiz == NULL)
        return;
    EscrevePosOrdem(Raiz->Esq);
    EscrevePosOrdem(Raiz->Dir);
    printf(" %c ",Raiz->info);
}
```



▶ 1 2 3 4 5 6 7 8 9

- ▶ Esvaziar uma árvore
- ▶ Liberta todo o espaço de memória da árvore e retorna a árvore ao estado equivalente ao define, isto é, nula
- ▶ Utiliza o algoritmo de Pós-Ordem para percurso.

```
void esvaziar(tarvbin *T)
    if (*T == NULL) return;

    esvaziar(&(*T)->esq);
    esvaziar(&(*T)->dir);
    free(*T);
    *T = NULL;
}
```


- ▶ Esvaziar uma árvore
- ▶ Liberta todo o espaço de memória da árvore e retorna a árvore ao estado equivalente ao define, isto é, nula
- ▶ Utiliza o algoritmo de Pós-Ordem para percurso.

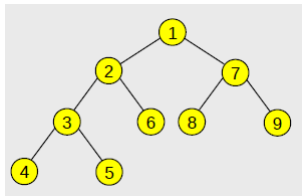
```
void esvaziar(tarvbin *T)
    if (*T == NULL) return;

    esvaziar(&(*T)->esq);
    esvaziar(&(*T)->dir);
    free(*T);
    *T = NULL;
}
```

- ▶ Esvaziar uma árvore
- ▶ Liberta todo o espaço de memória da árvore e retorna a árvore ao estado equivalente ao define, isto é, nula
- ▶ Utiliza o algoritmo de Pós-Ordem para percurso.

```
void esvaziar(tarvbin *T)
    if (*T == NULL) return;

    esvaziar(&(*T)->esq);
    esvaziar(&(*T)->dir);
    free(*T);
    *T = NULL;
}
```



► Método InserirNo (versão recursiva): Ex.: Inserir o «I»

```
void InserirNoRecursivo(tarvbin *Raiz, telem Item){  
  
    if (*Raiz==NULL){//Se a árvore estiver vazia  
        *Raiz= (tno*) malloc(sizeof(tno));  
        (*Raiz)->info=Item;  
        (*Raiz)->esq=NULL;  
        (*Raiz)->dir=NULL;  
        return;  
    }  
    if(Item > (*Raiz)->info)  
        InserirNoRecursivo(&(*Raiz)->dir, Item);  
    else  
        InserirNoRecursivo(&(*Raiz)->esq, Item);  
}
```

► Método InserirNo (versão recursiva): Ex.: Inserir o «I»

```
void InserirNoRecursivo(tarvbin *Raiz, telem Item){  
  
    if (*Raiz==NULL){//Se a árvore estiver vazia  
        *Raiz= (tno*) malloc(sizeof(tno));  
        (*Raiz)->info=Item;  
        (*Raiz)->esq=NULL;  
        (*Raiz)->dir=NULL;  
        return;  
    }  
    if(Item > (*Raiz)->info)  
        InserirNoRecursivo(&(*Raiz)->dir, Item);  
    else  
        InserirNoRecursivo(&(*Raiz)->esq, Item);  
}
```

► Método InserirNo (versão recursiva): Ex.: Inserir o «I»

```
void InserirNoRecursivo(tarvbin *Raiz, telem Item){
```

```
    if (*Raiz==NULL){//Se a árvore estiver vazia
```

```
        *Raiz= (tno*) malloc(sizeof(tno));
```

```
        (*Raiz)->info=Item;
```

```
        (*Raiz)->esq=NULL;
```

```
        (*Raiz)->dir=NULL;
```

```
        return;
```

```
    }
```

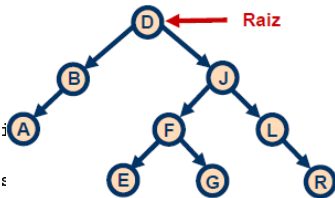
```
    if(Item > (*Raiz)->info)
```

```
        InserirNoRecursivo(&(*Raiz)->dir, Item);
```

```
    else
```

```
        InserirNoRecursivo(&(*Raiz)->esq, Item);
```

```
}
```



► Método InserirNo (versão recursiva): Ex.: Inserir o «I»

```
void InserirNoRecursivo(tarvbin *Raiz, telem Item){
```

```
    if (*Raiz==NULL){//Se a árvore estiver vazia
```

```
        *Raiz= (tno*) malloc(sizeof(tno));
```

```
        (*Raiz)->info=Item;
```

```
        (*Raiz)->esq=NULL;
```

```
        (*Raiz)->dir=NULL;
```

```
        return;
```

```
    }
```

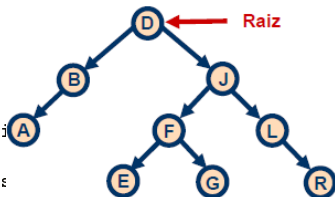
```
    if(Item > (*Raiz)->info)
```

```
        InserirNoRecursivo(&(*Raiz)->dir, Item);
```

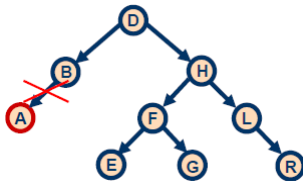
```
    else
```

```
        InserirNoRecursivo(&(*Raiz)->esq, Item);
```

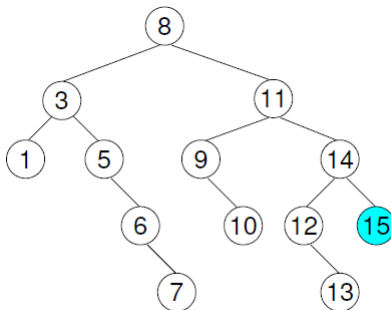
```
}
```



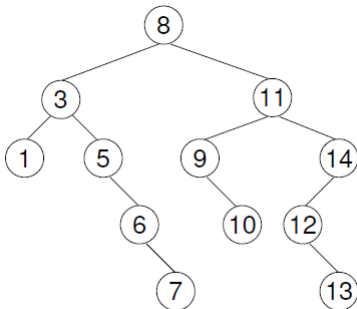
- ▶ Método EliminaNo:
 - ▶ A eliminação de um nó numa árvore binária é uma operação bastante mais complexa do que a inserção, uma vez que, grande parte das vezes, a eliminação de um nó implica a reorganização de toda a árvore para manter a ordem dos seus elementos.
- ▶ Existem três casos a considerar:
 - 1) O nó a eliminar é uma folha. Fácil de resolver: remover o nó da árvore fazendo o ponteiro no seu pai igual a NULL.



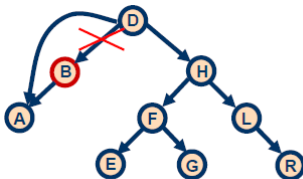
- ▶ Exemplo
 - ▶ Removendo a chave 15



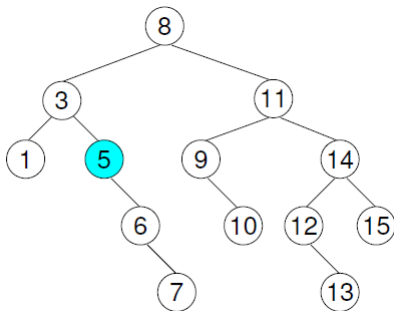
- ▶ Exemplo
 - ▶ Removendo a chave 15



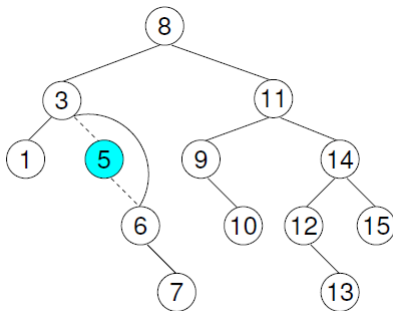
- ▶ Método EliminaNo:
- 2) O nó a eliminar tem apenas um único filho. Fácil de resolver: substituir o nó a eliminar pelo seu filho.



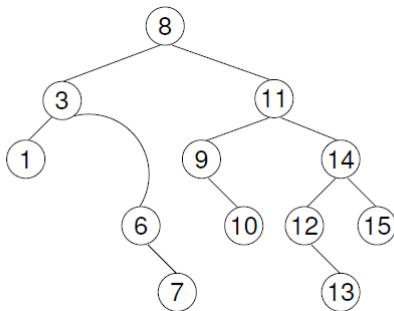
- ▶ Exemplo
 - ▶ Removendo a chave 5



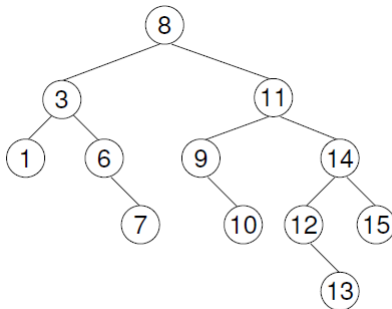
- ▶ Exemplo
 - ▶ Removendo a chave 5



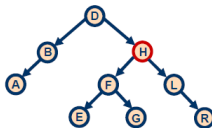
- ▶ Exemplo
 - ▶ Removendo a chave 5



- ▶ Exemplo
- ▶ ▶ Removendo a chave 5

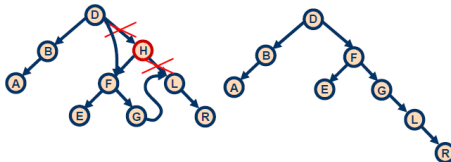


- ▶ Método EliminaNo:
- 3) O nó a eliminar tem dois filhos. Mais difícil de resolver: ambas as sub-árvores filhas do nó a eliminar tem de ser religadas à árvore de forma a manter a ordem correcta de todos os elementos. São sugeridas duas aproximações diferentes:
 - ▶ a) Substituir o nó a eliminar por um dos seus filhos e depois religar o outro filho na ordem correcta.
 - ▶ b) Substituir o nó a eliminar pelo seu descendente imediatamente a seguir.



► Método EliminaNo:

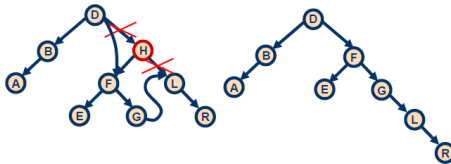
- 3 -a) Substituir o nó a eliminar por um dos seus filhos e depois religar o outro filho na ordem correcta.



Qual o problema desta aproximação??

► Método EliminaNo:

- 3 -a) Substituir o nó a eliminar por um dos seus filhos e depois religar o outro filho na ordem correcta.

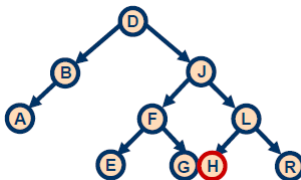
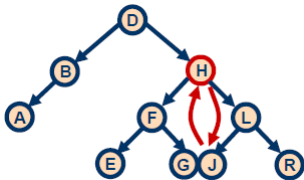


Qual o problema desta aproximação??

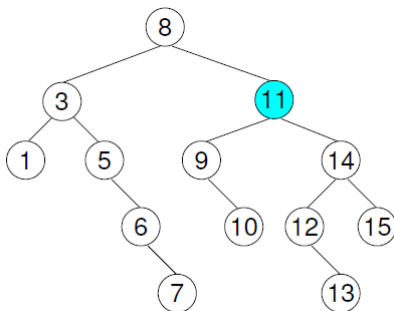
Resposta: **Árvore desequilibrada!!!**

► Método EliminaNo:

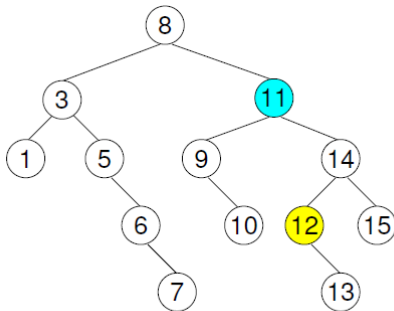
- 3 -b) Substituir o nó a eliminar pelo seu descendente imediatamente a seguir.
- Trocar o elemento a eliminar pelo seu sucessor na ordenação.
 - Eliminar o nó pretendido como se tratasse de um novo processo de eliminação. De notar que na pior das hipóteses o elemento a eliminar terá apenas um sucessor à direita.



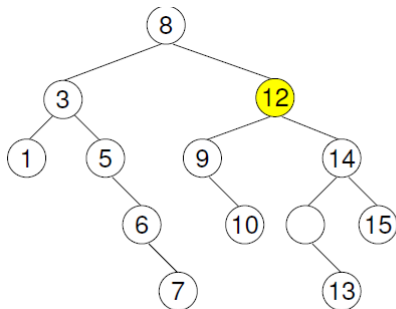
► **Exemplo:**Removendo a chave 11



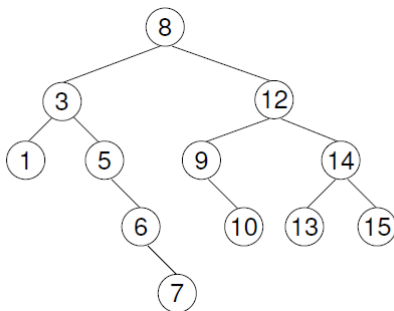
► **Exemplo:**Removendo a chave 11



► **Exemplo:**Removendo a chave 11



► **Exemplo:**Removendo a chave 11



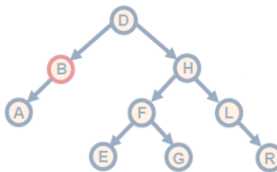
```
int EliminaNo(tarvbin *Raiz, telem Item){
    tno *Pai, *Aux, *Filho, * Sucessor;
    int Encontrado=0;
    Pai=NULL;
    Aux=*Raiz;
    while ((Aux != NULL) && ( !Encontrado)){
        if (Aux->info == Item)
            Encontrado = 1;
        else
            if (Item < Aux->info){
                Pai=Aux;
                Aux = Aux->esq;
            }
            else{
                Pai=Aux;
                Aux = Aux->dir;
            }
    }
}
```

Procura o
elemento a
eliminar

- ▶ Se o elemento foi Encontrado então Aux aponta para o elemento a eliminar e Pai aponta para o seu Pai.

```
if (Encontrado){  
    if ((Aux->esq==NULL) || (Aux->dir==NULL)) { // Nó tem no máximo um filho  
        if (Aux->esq != NULL)  
            Filho = Aux->esq;  
        else  
            Filho = Aux->dir;  
        if (Pai == NULL)  
            *Raiz = Filho; // Caso especial: apaga a raiz  
        else{ // Atualiza ligações  
            if (Aux->info < Pai->info)  
                Pai->esq = Filho;  
            else  
                Pai->dir = Filho;  
        }  
        free(Aux);  
    }  
}
```

Elimina nós
com zero ou
com um filho




```
else{    // Eliminar nó com dois filhos
    Sucessor= Aux->dir; Pai = Aux;
    while (Sucessor->esq != NULL){
        Pai = Sucessor;
        Sucessor = Sucessor->esq;
    }
    Aux->info = Sucessor->info;
    if (Sucessor == Aux->dir)
        Aux->dir = Sucessor->dir;
    else    // Substituir o nó com o seu filho à direita
        Pai->esq = Sucessor->dir;
        free(Sucessor);
}
return 1;
}
return 0;
}
```

Procura o seu
sucessor em
termos de
ordem

// Troca os valores

Elimina o nó
com a
informação a
eliminar

